Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de la Información
Boca del Monte, Villa Canales
Programación 1
Sección B

PROYECTO NO.1

TOTITO (3 EN RAYA)

ESTUDIANTE	**CARNÉ**
Bryan Alejandro Osoy Monterroso	7690-25-15671

DESCRIPCION DEL PROYECTO

¿Qué es el proyecto ?: El proyecto consiste en la implementación del juego clásico Totito (conocido también como 3 en raya) sobre una matriz ampliada de 10 filas por 10 columnas. A diferencia del tablero tradicional de 3×3, este diseño ofrece mayor espacio de juego y obliga a los jugadores a desarrollar estrategias más elaboradas para ganar.

El sistema está diseñado para dos jugadores que se turnan en la misma computadora para colocar sus símbolos en el tablero. El primero que logre alinear tres símbolos consecutivos — ya sea en horizontal, vertical o diagonal — gana la partida.

Herramientas Utilizadas: El programa fue desarrollado en el lenguaje de programación Java, utilizando el paradigma de Programación Orientada a Objetos (POO). Esto significa que el código está organizado en clases independientes, cada una con una responsabilidad clara y definida. Las clases se comunican entre sí a través de objetos, lo que hace el código más ordenado, reutilizable y fácil de entender.

Para la lectura de datos desde el teclado se utiliza la clase Scanner del paquete java.util, y para el manejo de archivos se utilizan las clases FileWriter, BufferedWriter, FileReader y BufferedReader del paquete java.io.

Funcionalidades Implementadas:

- Tablero de 10×10 con visualización numerada en filas y columnas
- Dos jugadores humanos con nombres y símbolos personalizados (no limitado a X y O)
- Detección de victoria en horizontal, vertical y ambas diagonales
- Detección de empate cuando el tablero se llena sin ganador
- Guardado de partida en archivo de texto (partida_guardada.txt)
- Restauración de partida guardada para continuar donde se dejó
- Registro de puntajes acumulados durante la sesión de juego
- Opción de reiniciar el juego con los mismos o nuevos jugadores
- Nombre y carné del estudiante visibles debajo del tablero en todo momento

OBJETIVOS

Objetivo General: Desarrollar un sistema de juego Totito funcional en lenguaje Java sobre un tablero de 10×10, aplicando correctamente los principios de la Programación Orientada a Objetos: encapsulamiento, abstracción y modularidad, con capacidad de persistir el estado del juego en archivos de texto.

Objetivos Específicos:

1. Implementar la lógica del juego Totito en una matriz bidimensional de 10 filas por 10 columnas utilizando arreglos de tipo char en Java.

2. Diseñar e implementar al menos cuatro clases bien diferenciadas (Jugador, Tablero, Partida y Main) siguiendo el principio de responsabilidad única de la POO.

3. Permitir que cada jugador elija su propio símbolo de forma libre (X, O u otro carácter), validando que no se repitan entre jugadores.

4. Implementar los algoritmos de detección de victoria en las cuatro direcciones posibles: horizontal, vertical, diagonal descendente y diagonal ascendente.

5. Desarrollar la funcionalidad de guardar el estado completo de una partida en un archivo de texto y de restaurarla posteriormente para continuar el juego.

6. Mantener un registro de puntajes entre partidas durante la misma sesión de ejecución del programa.

7. Mostrar de forma permanente el nombre y carné del estudiante debajo del tablero, cumpliendo el requisito establecido en el enunciado del proyecto.

DESCRIPCIONES DE CLASES, PROPIEDADES Y METODOS

Clase: Jugador

Descripción: Representa a uno de los dos participantes del juego. Almacena su identidad (nombre y símbolo) y lleva el conteo de partidas ganadas durante la sesión.

Atributo / Propiedad	Tipo	Descripción
`nombre`	String	Almacena el nombre completo del jugador tal como lo ingresó al inicio.
`simbolo`	char	Carácter único que representa al jugador en el tablero (ej: 'X', 'O', '*').
`puntaje`	int	Contador de partidas ganadas. Inicia en 0 y se incrementa con sumarPunto().

Método	Retorna	Descripción
`Jugador(nombre, simbolo)`	—	Constructor. Inicializa nombre y símbolo. El puntaje comienza en 0 automáticamente.
`getNombre()`	String	Retorna el nombre del jugador almacenado en el atributo nombre.
`setNombre(nombre)`	void	Permite cambiar el nombre del jugador. Útil al reiniciar partida con nuevos datos.
`getSimbolo()`	char	Retorna el símbolo del jugador que se usa para marcar casillas en el tablero.
`setSimbolo(simbolo)`	void	Permite cambiar el símbolo del jugador. Se usa al reiniciar con datos diferentes.
`getPuntaje()`	int	Retorna el número de partidas ganadas por este jugador en la sesión actual.
`sumarPunto()`	void	Incrementa en 1 el puntaje del jugador. Se llama automáticamente cuando gana una partida.
`toString()`	String	Retorna un texto con el nombre, símbolo y puntaje del jugador para mostrar en pantalla.

CLASE TABLERO

Clase: Tablero		
Descripción: Es el núcleo del juego. Maneja la matriz de 10×10, controla el estado de cada casilla, muestra el tablero en consola y contiene los algoritmos para detectar victoria o empate.		
Atributo / Propiedad	**Tipo**	**Descripción**
matriz	char[][]	Arreglo bidimensional de 10×10 char. Cada celda contiene '.' si está vacía, o el símbolo del jugador que la ocupó.
filas	int	Constante con valor 10. Define el número de filas de la matriz.
columnas	int	Constante con valor 10. Define el número de columnas de la matriz.
Método	**Retorna**	**Descripción**
Tablero()	—	Constructor. Crea la matriz de 10×10 y llama a limpiarTablero() para inicializarla.
limpiarTablero()	void	Recorre toda la matriz con dos bucles for anidados y coloca '.' en cada celda, indicando que está vacía.
mostrarTablero(nombre, carnet)	void	Imprime la matriz en consola con encabezado de columnas (1–10) y numeración de filas. Al final muestra el nombre y carnet del estudiante.
colocarSimbolo(fila, col, simbolo)	boolean	Verifica que fila y columna estén en rango (0–9) y que la celda esté vacía ('.'). Si todo es válido, coloca el símbolo y retorna true. Si no, retorna false.

hayGanador(simbolo)	boolean	Algoritmo principal de victoria. Revisa las 4 direcciones (horizontal, vertical, diagonal y diagonal) buscando 3 símbolos consecutivos iguales. Retorna true si los encuentra.
tableroLleno()	boolean	Recorre toda la matriz buscando alguna celda vacía ('.'). Si no encuentra ninguna, retorna true (empate).
getMatriz()	char[][]	Retorna la referencia a la matriz para que la clase Partida pueda leerla y guardarla en archivo.
setMatriz(nuevaMatriz)	void	Reemplaza la matriz actual con una previamente cargada desde archivo. Usado al restaurar partida.

UNIVERSIDAD RAFAEL LANDIVAR
CONOCEREIS LA VERDAD Y LA VERDAD OS HARA LIBRES
GUATEMALA

CLASE PARTIDA

Clase: Partida		
Descripción: Se encarga de toda la persistencia del juego. Permite guardar el estado completo de una partida en un archivo de texto y posteriormente cargarlo para continuar jugando.		
Atributo / Propiedad	**Tipo**	**Descripción**
nombreArchivo	String	Nombre del archivo de texto donde se persiste la partida. Valor fijo: 'partida_guardada.txt'. Se crea en el directorio de ejecución.
Método	**Retorna**	**Descripción**
guardarPartida(j1, j2, matriz, turno)	void	Abre el archivo en modo escritura y escribe en orden: nombre/símbolo/puntaje del J1, nombre/símbolo/puntaje del J2, turno actual, y las 10 filas del tablero. Cada dato en una línea separada.
cargarPartida()	String[]	Lee el archivo línea por línea y retorna un arreglo de 17 Strings: 7 datos de jugadores y turno, más 10 líneas del tablero. Retorna null si ocurre un error.
borrarPartida()	void	Elimina el archivo físico del disco. Se llama automáticamente cuando una partida termina (victoria o empate) para no restaurar una partida ya finalizada.
existePartida()	boolean	Comprueba si el archivo 'partida_guardada.txt' existe en el sistema de archivos. Retorna true si hay una partida guardada disponible para restaurar.

CLASE MAIN

Clase: Main

Descripción: Es la clase principal del programa. Contiene el método main() que inicia todo y orquesta la interacción entre las demás clases. Maneja el menú, los turnos, la validación de jugadas y el flujo general del juego.

Atributo / Propiedad	Tipo	Descripción
`sc`	Scanner	Objeto global para leer la entrada del teclado en cualquier método de la clase.
`tablero`	Tablero	Instancia del tablero de juego. Se reutiliza entre partidas limpiándolo con limpiarTablero().
`partida`	Partida	Instancia de la clase Partida para acceder a guardar/cargar en cualquier momento.
`jugador1`	Jugador	Objeto del primer jugador. Se crea al configurar nuevo juego y persiste entre rondas para mantener el puntaje.
`jugador2`	Jugador	Objeto del segundo jugador. Igual que jugador1, persiste entre rondas de la misma sesión.
`turnoActual`	int	Indica qué jugador tiene el turno: 1 para jugador1, 2 para jugador2. Se alterna después de cada jugada válida.
`nombreEstudiante`	String	Nombre del estudiante que se muestra debajo del tablero. Debe ser reemplazado con el nombre real.
`carnetEstudiante`	String	Carnet del estudiante que se muestra debajo del tablero. Debe ser reemplazado con el carnet real.

Método	Retorna	Descripción
`main(args)`	void	Punto de entrada del programa. Muestra el mensaje de bienvenida y llama a menuPrincipal() para iniciar el flujo.
`menuPrincipal()`	void	Muestra las 4 opciones del menú principal en un bucle while. Procesa la opción seleccionada con un switch. Continúa hasta que el usuario elige salir (opción 4).

`configurarNuevoJuego()`	void	Solicita nombres y símbolos a ambos jugadores. Si ya hubo partidas anteriores, pregunta si se desean reiniciar los puntajes. Limpia el tablero y llama a iniciarJuego().
`pedirSimbolo(n, otro)`	char	Solicita un símbolo al jugador y valida que: no sea el carácter '.', no esté vacío, y no sea igual al símbolo del otro jugador. Repite hasta recibir un símbolo válido.
`iniciarJuego()`	void	Bucle principal del juego. En cada iteración muestra el tablero, el menú de turno, solicita la jugada, valida y coloca el símbolo, luego verifica si hay ganador o empate.
`preguntarOtraPartida()`	void	Al terminar una partida, pregunta si desean jugar otra ronda. Si el usuario acepta, limpia el tablero y llama de nuevo a iniciarJuego().
`restaurarPartida()`	void	Verifica si hay archivo guardado, lo carga con la clase Partida, reconstruye los objetos Jugador y la matriz del Tablero, y continúa el juego con iniciarJuego().
`mostrarPuntajes()`	void	Imprime en consola el toString() de ambos jugadores mostrando nombre, símbolo y puntaje acumulado.

FUNCIONALIDADES DE METODOS IMPORTANTES

hayGanador() — Clase Tablero: Este es el método más crítico del juego. Se ejecuta después de cada jugada para determinar si el jugador que acaba de mover ha ganado la partida. Recibe el símbolo del jugador actual y recorre la matriz buscando tres casillas consecutivas con ese mismo símbolo en cuatro direcciones diferentes:

HORIZONTAL: Revisa cada fila (i=0..9), en cada posición de columna (j=0..7) compara matriz[i][j], [i][j+1], [i][j+2].

VERTICAL: Revisa cada columna (j=0..9), en cada posición de fila (i=0..7) compara matriz[i][j], [i+1][j], [i+2][j].

DIAGONAL ↘: Desde (i=0..7, j=0..7) compara [i][j], [i+1][j+1], [i+2][j+2].

DIAGONAL ↙: Desde (i=0..7, j=2..9) compara [i][j], [i+1][j-1], [i+2][j-2].

El método retorna true en cuanto encuentra la primera combinación ganadora, sin necesidad de seguir revisando. Si recorre toda la matriz sin encontrar tres en línea, retorna false y el juego continúa.

guardarPartida() — Clase Partida: Este método serializa manualmente todo el estado del juego en un archivo de texto plano. Utiliza FileWriter y BufferedWriter para escribir cada dato en una línea separada. El orden exacto de escritura es fundamental, ya que cargarPartida() debe leer los datos en el mismo orden para restaurarlos correctamente.

Línea 1: Nombre del Jugador 1

Línea 2: Símbolo del Jugador 1

Línea 3: Puntaje del Jugador 1

Línea 4: Nombre del Jugador 2

Línea 5: Símbolo del Jugador 2

Línea 6: Puntaje del Jugador 2

Línea 7: Turno actual (1 o 2)

Líneas 8–17: Las 10 filas del tablero (cada fila como una cadena de 10 caracteres)

Al terminar una partida (victoria o empate), el archivo es eliminado automáticamente por borrarPartida() para que no se pueda restaurar una partida que ya concluyó.

colocarSimbolo() — Clase Tablero:

Este método actúa como validador y ejecutor de cada jugada. Antes de colocar el símbolo en la matriz, realiza dos verificaciones esenciales:

1**. Rango válido:** comprueba que la fila esté entre 0 y 9 y que la columna esté entre 0 y 9. Esto evita ArrayIndexOutOfBoundsException en Java.

2. **Casilla libre:** verifica que la celda seleccionada contenga '.' (vacía). Si ya tiene un símbolo, significa que ya fue jugada y no se puede repetir.

Si ambas verificaciones pasan, el símbolo se coloca en la posición indicada y el método retorna true. Si falla cualquiera, retorna false y el método iniciarJuego() en Main le muestra al usuario un mensaje de error y le pide que ingrese otra posición.

restaurarPartida() — Clase Main:

Este método reconstruye completamente el estado del juego a partir de los datos guardados en el archivo de texto. El proceso paso a paso es:

Paso 1: Verifica que exista el archivo llamando a existePartida(). Si no existe, informa al usuario y regresa al menú.

Paso 2: Llama a cargarPartida() que lee el archivo y retorna un arreglo de 17 Strings con todos los datos.

Paso 3: Crea nuevos objetos Jugador usando datos[0] como nombre y datos[1].charAt(0) como símbolo para el jugador 1. Igual para jugador 2 con datos[3] y datos[4].

Paso 4: Restaura los puntajes haciendo un bucle que llama a sumarPunto() tantas veces como indique el valor guardado.

Paso 5: Restaura el turnoActual parseando datos[6] como entero.

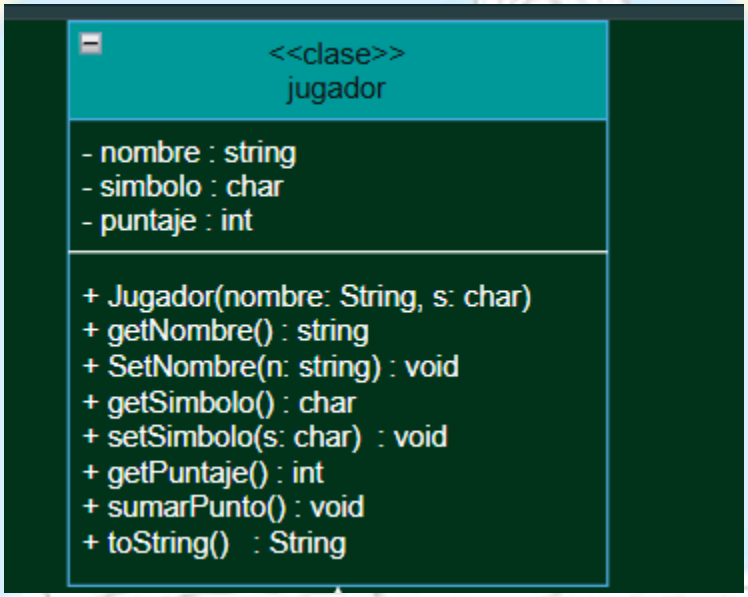
Paso 6: Reconstruye la matriz de 10×10 leyendo los caracteres de las líneas datos[7] a datos[16], asignando cada char a su posición correspondiente.

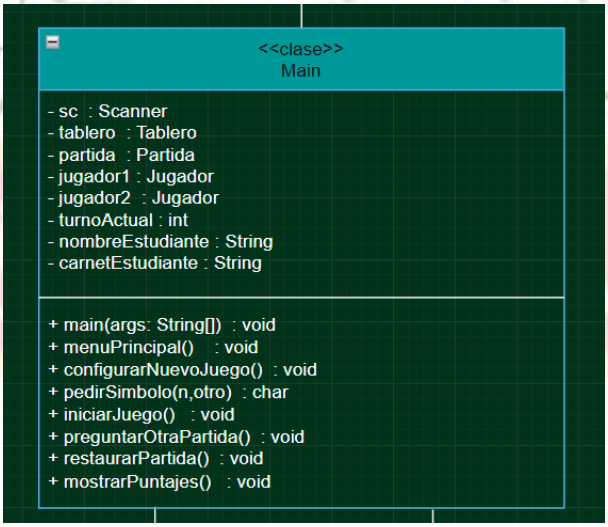
Paso 7: Llama a iniciarJuego() para continuar desde el punto exacto donde se guardó.

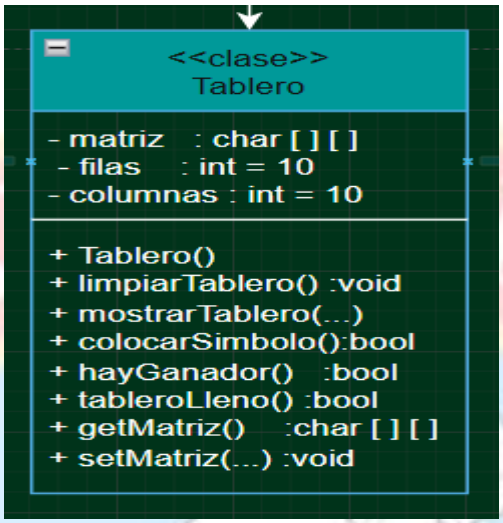
DIAGRAMA DE CLASES

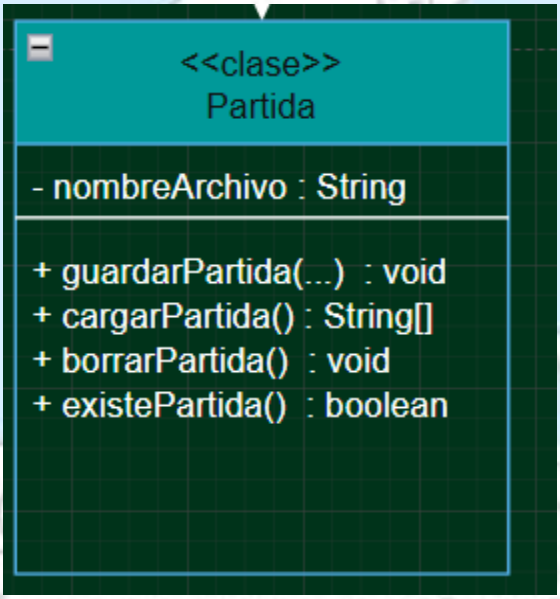
El siguiente diagrama muestra la estructura completa del sistema y las relaciones de uso (dependencia) entre las clases. La clase Main es el punto central que crea y coordina a las demás clases.

Enlace:
https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=_blank&layers=1&nav=1&title=Diagrama%20sin%20t%C3%ADtulo.drawio&dark=auto#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D11xwVS0VJosCYYZ61jbkjbtXk-H9W6QjZ%26export%3Ddownload









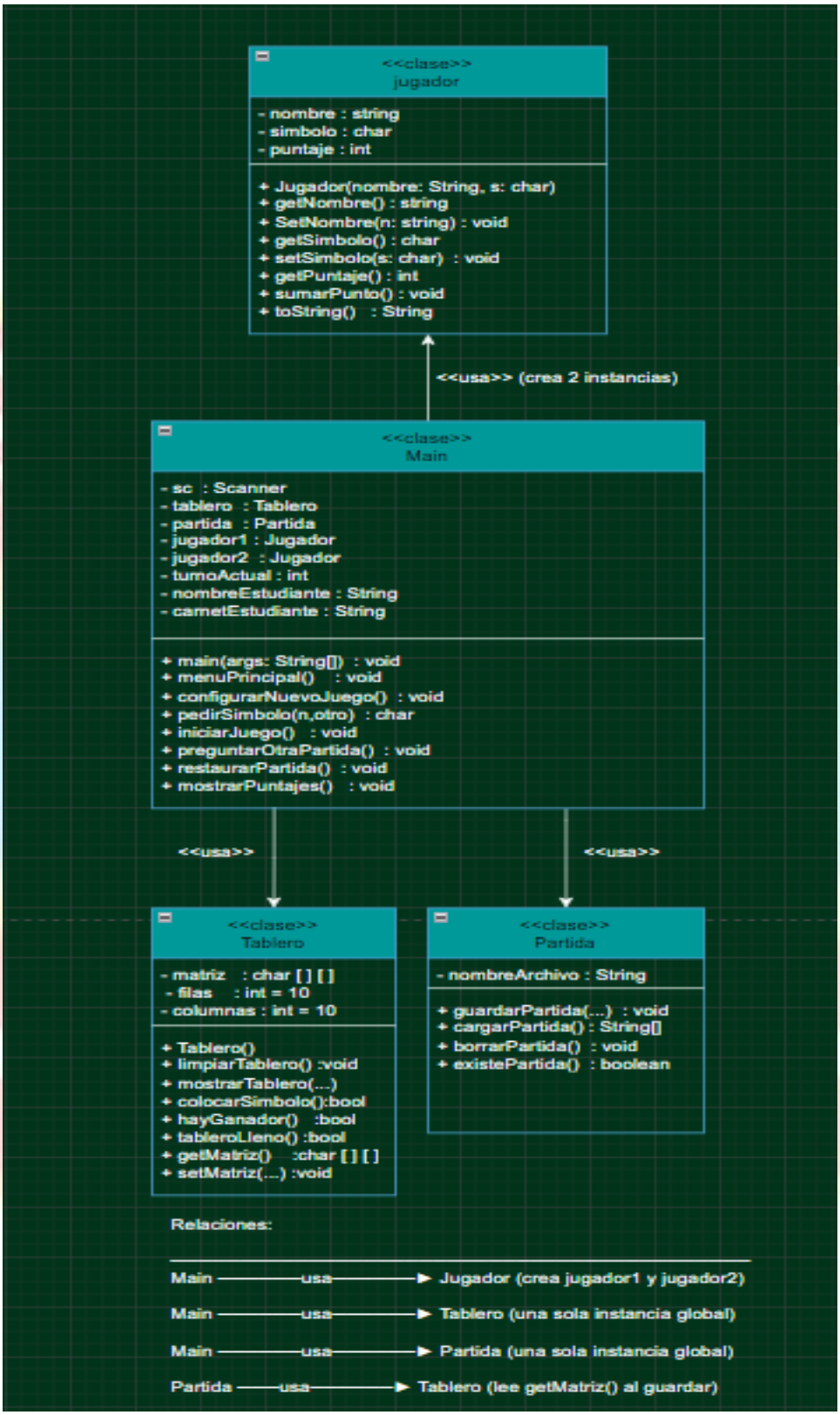
Relaciones:

Main ——usa——▶ Jugador (crea jugador1 y jugador2)

Main ——usa——▶ Tablero (una sola instancia global)

Main ——usa——▶ Partida (una sola instancia global)

Partida ——usa——▶ Tablero (lee getMatriz() al guardar)


<<clase>>
jugador
- nombre : string
- simbolo : char
- puntaje : int
+ Jugador(nombre: String, s: char)
+ getNombre() : string
+ SetNombre(n: string) : void
+ getSimbolo() : char
+ setSimbolo(s: char) : void
+ getPuntaje() : int
+ sumarPunto() : void
+ toString() : String
<<usa>> (crea 2 instancias)
<<clase>>
Main
- sc : Scanner
- tablero : Tablero
- partida : Partida
- jugador1 : Jugador
- jugador2 : Jugador
- turnoActual : int
- nombreEstudiante : String
- cametEstudiante : String
+ main(args: String[]) : void
+ menuPrincipal() : void
+ configurarNuevoJuego() : void
+ pedirSimbolo(n,otro) : char
+ iniciarJuego() : void
+ preguntarOtraPartida() : void
+ restaurarPartida() : void
+ mostrarPuntajes() : void
<<usa>>
<<usa>>
<<clase>>
Tablero
- matriz : char [ ] [ ]
- filas : int = 10
- columnas : int = 10
+ Tablero()
+ limpiarTablero() :void
+ mostrarTablero(...)
+ colocarSimbolo():bool
+ hayGanador() :bool
+ tableroLleno() :bool
+ getMatriz() :char [ ] [ ]
+ setMatriz(...) :void
<<clase>>
Partida
- nombreArchivo : String
+ guardarPartida(...) : void
+ cargarPartida() : String[]
+ borrarPartida() : void
+ existePartida() : boolean
Relaciones:
Main ——usa——► Jugador (crea jugador1 y jugador2)
Main ——usa——► Tablero (una sola instancia global)
Main ——usa——► Partida (una sola instancia global)
Partida ——usa——► Tablero (lee getMatriz() al guardar)

EXPLICACION DE RELACIONES

Main → Jugador (dependencia de creación): La clase Main declara dos atributos estáticos de tipo Jugador (jugador1 y jugador2). Los crea mediante el constructor new Jugador(nombre, simbolo) en el método configurarNuevoJuego(). Main es la única clase que accede directamente a los métodos de Jugador.

Main → Tablero (dependencia de uso): Main declara un atributo estático de tipo Tablero. Llama a sus métodos para mostrar el tablero, colocar símbolos, verificar ganador y limpiar al iniciar nuevas rondas.

Main → Partida (dependencia de uso): Main declara un atributo estático de tipo Partida. Lo utiliza para guardar la partida cuando el usuario lo solicita y para restaurarla desde el menú principal.

Partida → Tablero (dependencia indirecta): Al guardar, Main le pasa la matriz obtenida de tablero.getMatriz() como parámetro al método guardarPartida(). Al cargar, Main le pasa la matriz reconstruida a tablero.setMatriz(). Partida no tiene una referencia directa a Tablero, sino que trabaja con el arreglo char[][].

CODIGO FUENTE EN JAVA

A continuación se presenta el código fuente completo de cada clase del proyecto.

CLASE JUGADOR:

```java
// Clase que representa a un jugador del juego
// Autor: [Bryan Alejandro Osoy Monterroso] - Carnet: [7690 25 15671]

public class Jugador {

    private String nombre;
    private char simbolo;
    private int puntaje;

    // Constructor del jugador
    public Jugador(String nombre, char simbolo) {
        this.nombre = nombre;
        this.simbolo = simbolo;
        this.puntaje = 0;
    }

    // metodos get y set
    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public char getSimbolo() {
        return simbolo;
    }

    public void setSimbolo(char simbolo) {
        this.simbolo = simbolo;
    }

    public int getPuntaje() {
        return puntaje;
    }

    public void sumarPunto() {
        this.puntaje++;
    }

    // para mostrar info del jugador
    public String toString() {
        return nombre + " (" + simbolo + ") - Puntos: " + puntaje;
    }
}
```

CLASE TABLERO:

```java
// Clase que maneja el tablero de 10x10 y la logica del juego
// Autor: [Bryan Alejandro Osoy Monterroso] - Carnet: [7690 25 15671]

public class Tablero {

    // la matriz del juego es de 10x10
    private char[][] matriz;
    private int filas = 10;
    private int columnas = 10;

    public Tablero() {
        matriz = new char[filas][columnas];
        limpiarTablero();
    }

    // limpia el tablero poniendo espacios vacios
    public void limpiarTablero() {
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                matriz[i][j] = '.';
            }
        }
    }

    // muestra el tablero en consola
    public void mostrarTablero(String nombreJ1, String carnetJ1) {
        System.out.println("\n   1 2 3 4 5 6 7 8 9 10");
        for (int i = 0; i < filas; i++) {
            // para alinear bien los numeros de fila
            if (i < 9) {
                System.out.print((i + 1) + "  ");
            } else {
                System.out.print((i + 1) + " ");
            }
            for (int j = 0; j < columnas; j++) {
                System.out.print(matriz[i][j] + " ");
            }
            System.out.println();
        }
        // mostrar nombre y carnet debajo del tablero
        System.out.println("\n" + nombreJ1 + " - " + carnetJ1);
    }

    // coloca el simbolo en la posicion indicada
    public boolean colocarSimbolo(int fila, int columna, char simbolo) {
        // verificamos que la casilla este libre
        if (fila >= 0 && fila < filas && columna >= 0 && columna < columnas) {
            if (matriz[fila][columna] == '.') {
                matriz[fila][columna] = simbolo;
                return true;
            }
        }
        return false;
    }

    // revisa si alguien gano
    public boolean hayGanador(char simbolo) {
        // revisar filas
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j <= columnas - 3; j++) {
                if (matriz[i][j] == simbolo && matriz[i][j+1] == simbolo && matriz[i][j+2] == simbolo) {
                    return true;
                }
            }
        }

        // revisar columnas
        for (int i = 0; i <= filas - 3; i++) {
            for (int j = 0; j < columnas; j++) {
                if (matriz[i][j] == simbolo && matriz[i+1][j] == simbolo && matriz[i+2][j] == simbolo) {
                    return true;
                }
            }
        }

```

```java
// revisar diagonal hacia abajo-derecha
        for (int i = 0; i <= filas - 3; i++) {
            for (int j = 0; j <= columnas - 3; j++) {
                if (matriz[i][j] == simbolo && matriz[i+1][j+1] == simbolo && matriz[i+2][j+2] == simbolo) {
                    return true;
                }
            }
        }

        // revisar diagonal hacia abajo-izquierda
        for (int i = 0; i <= filas - 3; i++) {
            for (int j = 2; j < columnas; j++) {
                if (matriz[i][j] == simbolo && matriz[i+1][j-1] == simbolo && matriz[i+2][j-2] == simbolo) {
                    return true;
                }
            }
        }

        return false;
    }

    // revisa si el tablero esta lleno (empate)
    public boolean tableroLleno() {
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                if (matriz[i][j] == '.') {
                    return false;
                }
            }
        }
        return true;
    }

    // devuelve la matriz para poder guardarla
    public char[][] getMatriz() {
        return matriz;
    }

    // carga una matriz guardada
    public void setMatriz(char[][] nuevaMatriz) {
        this.matriz = nuevaMatri
```

PARTIDA:

```java
// Clase para guardar y restaurar partidas en archivo de texto
// Autor: [Bryan Alejandro Osoy Monterroso] - Carnet: [7690 25 15671]

import java.io.*;

public class Partida {

    private String nombreArchivo = "partida_guardada.txt";

    // guarda la partida en un archivo de texto
    public void guardarPartida(Jugador j1, Jugador j2, char[][] matriz, int turnoActual) {
        try {
            FileWriter fw = new FileWriter(nombreArchivo);
            BufferedWriter bw = new BufferedWriter(fw);




            // guardamos los datos de los jugadores
            bw.write(j1.getNombre() + "\n");
            bw.write(j1.getSimbolo() + "\n");
            bw.write(j1.getPuntaje() + "\n");
            bw.write(j2.getNombre() + "\n");
            bw.write(j2.getSimbolo() + "\n");
            bw.write(j2.getPuntaje() + "\n");
            bw.write(turnoActual + "\n");

            // guardamos el tablero fila por fila
            for (int i = 0; i < 10; i++) {
                for (int j = 0; j < 10; j++) {
                    bw.write(matriz[i][j] + "");
                }
                bw.write("\n");
            }

            bw.close();
            fw.close();
            System.out.println("Partida guardada correctamente!");

        } catch (IOException e) {
            System.out.println("Error al guardar la partida: " + e.getMessage());
        }
    }

    // carga la partida del archivo
    // devuelve null si no hay partida guardada
    public String[] cargarPartida() {
        File archivo = new File(nombreArchivo);

        if (!archivo.exists()) {
            System.out.println("No hay ninguna partida guardada.");
            return null;
        }

```

```java
try {
            FileReader fr = new FileReader(nombreArchivo);
            BufferedReader br = new BufferedReader(fr);

            // 7 datos de jugadores + turno, mas 10 filas del tablero = 17 lineas
            String[] datos = new String[17];
            for (int i = 0; i < 17; i++) {
                datos[i] = br.readLine();
            }

            br.close();
            fr.close();
            return datos;

        } catch (IOException e) {
            System.out.println("Error al cargar la partida: " + e.getMessage());
            return null;
        }
    }

    // borra el archivo de la partida guardada
    public void borrarPartida() {
        File archivo = new File(nombreArchivo);
        if (archivo.exists()) {
            archivo.delete();
        }
    }

    // revisa si hay una partida guardada
    public boolean existePartida() {
        File archivo = new File(nombreArchivo);
        return archivo.exists();
    }
}

```

MAIN:

```java
// Clase principal
// Proyecto 01 - Totito (3 en raya) en matriz 10x10
// Autor: [BRYAN ALEJANDRO OSOY MONTESORRO] - Carnet: [7690 25 15671]

import java.util.Scanner;

public class Main {

    static Scanner sc = new Scanner(System.in);
    static Tablero tablero = new Tablero();
    static Partida partida = new Partida();
    static Jugador jugador1 = null;
    static Jugador jugador2 = null;
    // 1 para jugador1, 2 para jugador2
    static int turnoActual = 1;

    static String nombreEstudiante = "[Bryan Alejandro Osoy Montesorro]";
    static String carnetEstudiante = "[7690 25 15671]";

    public static void main(String[] args) {
        System.out.println("=== BIENVENIDO AL TOTITO (3 EN RAYA) ===");
        System.out.println("Tablero de 10x10");

        menuPrincipal();
    }

    // menu principal del programa
    public static void menuPrincipal() {
        int opcion = 0;

        while (opcion != 4) {
            System.out.println("\n--- MENU PRINCIPAL ---");
            System.out.println("1. Nuevo juego");
            System.out.println("2. Restaurar partida guardada");
            System.out.println("3. Ver puntajes");
            System.out.println("4. Salir");
            System.out.print("Elige una opcion: ");

            try {
                opcion = Integer.parseInt(sc.nextLine());
            } catch (NumberFormatException e) {
                System.out.println("Por favor ingresa un numero valido.");
                continue;
            }

            switch (opcion) {
                case 1:
                    configurarNuevoJuego();
                    break;
                case 2:
                    restaurarPartida();
                    break;
                case 3:
                    mostrarPuntajes();
                    break;
                case 4:
                    System.out.println("Gracias por jugar. Hasta luego!");
                    break;
                default:
                    System.out.println("Opcion no valida, intenta de nuevo.");
            }
        }
    }
```

```java
// configura los datos para empezar un juego nuevo
    public static void configurarNuevoJuego() {
        System.out.println("\n--- NUEVO JUEGO ---");

        // pedir nombre del jugador 1
        System.out.print("Nombre del Jugador 1: ");
        String nombre1 = sc.nextLine();

        // pedir simbolo del jugador 1
        char simbolo1 = pedirSimbolo(nombre1, ' ');

        // pedir nombre del jugador 2
        System.out.print("Nombre del Jugador 2: ");
        String nombre2 = sc.nextLine();

        // el jugador 2 no puede usar el mismo simbolo
        char simbolo2 = pedirSimbolo(nombre2, simbolo1);

        // si ya habia jugadores, preguntamos si reiniciar puntajes
        if (jugador1 != null && jugador2 != null) {
            System.out.print("Deseas reiniciar los puntajes? (s/n): ");
            String resp = sc.nextLine();
            if (resp.equalsIgnoreCase("s")) {
                jugador1 = new Jugador(nombre1, simbolo1);
                jugador2 = new Jugador(nombre2, simbolo2);
            } else {
                // actualizamos nombre y simbolo pero mantenemos puntaje
                jugador1.setNombre(nombre1);
                jugador1.setSimbolo(simbolo1);
                jugador2.setNombre(nombre2);
                jugador2.setSimbolo(simbolo2);
            }
        } else {
            jugador1 = new Jugador(nombre1, simbolo1);
            jugador2 = new Jugador(nombre2, simbolo2);
        }

        tablero.limpiarTablero();
        turnoActual = 1;
        iniciarJuego();
    }

    // pide el simbolo al jugador, evita que use el mismo que el otro
    public static char pedirSimbolo(String nombre, char simboloOtro) {
        char simbolo = ' ';
        boolean valido = false;

        while (!valido) {
            System.out.print("Simbolo para " + nombre + " (X, O u otro caracter): ");
            String entrada = sc.nextLine();

            if (entrada.length() == 0) {
                System.out.println("Debes ingresar al menos un caracter.");
                continue;
            }

            simbolo = entrada.charAt(0);

            // verificamos que no sea punto porque lo usamos para casillas vacias
            if (simbolo == '.') {
                System.out.println("El punto (.) esta reservado para casillas vacias. Elige otro.");
                continue;
            }

            // verificamos que no sea igual al otro jugador
            if (simbolo == simboloOtro) {
                System.out.println("Ese simbolo ya lo usa el otro jugador. Elige uno diferente.");
                continue;
            }

            valido = true;
        }

        return simbolo;
    }
```

```java
// aqui se juega la partida
    public static void iniciarJuego() {
        boolean juegoTerminado = false;

        while (!juegoTerminado) {
            Jugador jugadorActual;
            if (turnoActual == 1) {
                jugadorActual = jugador1;
            } else {
                jugadorActual = jugador2;
            }

            tablero.mostrarTablero(nombreEstudiante, carnetEstudiante);
            System.out.println("\nTurno de: " + jugadorActual.getNombre() + " [" + jugadorActual.getSimbolo() + "]");

            // menu durante el juego
            System.out.println("1. Hacer jugada");
            System.out.println("2. Guardar partida");
            System.out.println("3. Salir al menu principal");
            System.out.print("Opcion: ");

            int opcion;
            try {
                opcion = Integer.parseInt(sc.nextLine());
            } catch (NumberFormatException e) {
                System.out.println("Opcion no valida.");
                continue;
            }

            if (opcion == 2) {
                partida.guardarPartida(jugador1, jugador2, tablero.getMatriz(), turnoActual);
                continue;
            } else if (opcion == 3) {
                System.out.print("Deseas guardar antes de salir? (s/n): ");
                String resp = sc.nextLine();
                if (resp.equalsIgnoreCase("s")) {
                    partida.guardarPartida(jugador1, jugador2, tablero.getMatriz(), turnoActual);
                }
                return;
            } else if (opcion != 1) {
                System.out.println("Opcion no valida.");
                continue;
            }

            // pedir la jugada
            int fila = -1;
            int col = -1;

            try {
                System.out.print("Fila (1-10): ");
                fila = Integer.parseInt(sc.nextLine()) - 1;
                System.out.print("Columna (1-10): ");
                col = Integer.parseInt(sc.nextLine()) - 1;
            } catch (NumberFormatException e) {
                System.out.println("Ingresa numeros validos.");
                continue;
            }

            boolean jugadaOk = tablero.colocarSimbolo(fila, col, jugadorActual.getSimbolo());


            if (!jugadaOk) {
                System.out.println("Casilla ocupada o fuera de rango, intenta de nuevo.");
                continue;
            }
```

```java
// revisar si gano
            if (tablero.hayGanador(jugadorActual.getSimbolo())) {
                tablero.mostrarTablero(nombreEstudiante, carnetEstudiante);
                System.out.println("\n*** " + jugadorActual.getNombre() + " ha ganado! Felicidades! ***");
                jugadorActual.sumarPunto();
                partida.borrarPartida();
                juegoTerminado = true;
                preguntarOtraPartida();
            } else if (tablero.tableroLleno()) {
                tablero.mostrarTablero(nombreEstudiante, carnetEstudiante);
                System.out.println("\nEmpate! El tablero esta lleno.");
                partida.borrarPartida();
                juegoTerminado = true;
                preguntarOtraPartida();
            } else {
                // cambiar turno
                if (turnoActual == 1) {
                    turnoActual = 2;
                } else {
                    turnoActual = 1;
                }
            }
        }
    }

    // pregunta si quieren jugar otra ronda
    public static void preguntarOtraPartida() {
        System.out.print("\nDesean jugar otra partida con los mismos jugadores? (s/n): ");
        String resp = sc.nextLine();
        if (resp.equalsIgnoreCase("s")) {
            tablero.limpiarTablero();
            turnoActual = 1;
            iniciarJuego();
        }
    }

    // restaura una partida guardada
    public static void restaurarPartida() {
        if (!partida.existePartida()) {
            System.out.println("No hay ninguna partida guardada.");
            return;
        }

        String[] datos = partida.cargarPartida();
        if (datos == null) return;

        // reconstruimos los jugadores con los datos guardados
        jugador1 = new Jugador(datos[0], datos[1].charAt(0));
        // no tenemos setter de puntaje, lo sumamos manualmente
        int pts1 = Integer.parseInt(datos[2]);
        for (int i = 0; i < pts1; i++) jugador1.sumarPunto();
```

```java
jugador2 = new Jugador(datos[3], datos[4].charAt(0));
        int pts2 = Integer.parseInt(datos[5]);
        for (int i = 0; i < pts2; i++) jugador2.sumarPunto();

        turnoActual = Integer.parseInt(datos[6]);

        // reconstruimos la matriz
        char[][] matrizCargada = new char[10][10];
        for (int i = 0; i < 10; i++) {
            for (int j = 0; j < 10; j++) {
                matrizCargada[i][j] = datos[7 + i].charAt(j);
            }
        }
        tablero.setMatriz(matrizCargada);

        System.out.println("Partida restaurada! Continua jugando...");
        iniciarJuego();
    }

    // muestra los puntajes actuales
    public static void mostrarPuntajes() {
        if (jugador1 == null) {
            System.out.println("Todavia no se ha jugado ninguna partida.");
            return;
        }
        System.out.println("\n--- PUNTAJES ---");
        System.out.println(jugador1.toString());
        System.out.println(jugador2.toString());
    }
}
```

MANUAL DE USUARIO

REQUISITOS PARA EJECUTAR EL PROGRAMA:

- Java JDK 8 o superior instalado en la computadora
- Una terminal o consola de comandos (cmd / powerShell / editor de texto)

Para verificar que Java está instalado, abre la terminal y escribe:

Java -version

COMO EJECUTAR EL PROGRAMA:

- Javac *.java
- Java Main

El programa se abrirá y mostrará el menú principal en la misma terminal.

MENU PRINCIPAL: Al iniciar el programa aparece este menú:

```
=== BIENVENIDO AL TOTITO (3 EN RAYA) ===
Tablero de 10x10

--- MENU PRINCIPAL ---
1. Nuevo juego
2. Restaurar partida guardada
3. Ver puntajes
4. Salir
Elige una opcion:
```

Escribe el número de la opción que deseas y presiona Enter

Opción	Nombre	¿Qué hace?
1	**Nuevo juego**	Inicia una partida nueva. Pide los nombres y símbolos de ambos jugadores.
2	**Restaurar partida**	Carga la última partida guardada y continúa desde donde se dejó.
3	**Ver puntajes**	Muestra cuántas partidas ha ganado cada jugador en la sesión actual.
4	**Salir**	Cierra el programa.

COMO INICAR UN NUEVO JUEGO: Cuando eliges la opción 1, el programa pide los datos de ambos jugadores uno por uno:

1	**Nombre del Jugador 1** Escribe el nombre completo y presiona Enter.
2	**Símbolo del Jugador 1** Escribe un carácter (X, O, *, # u otro). No puede ser el punto (.) porque está reservado para casillas vacías.
3	**Nombre del Jugador 2** Escribe el nombre del segundo jugador y presiona Enter.
4	**Símbolo del Jugador 2** Escribe un símbolo diferente al del Jugador 1. Si intentas usar el mismo, el programa te lo avisa y pide otro.

Si ya se había jugado antes, el programa preguntará si deseas reiniciar los puntajes (s/n).

Si eliges NO, se conservan los puntajes de la sesión anterior

COMO SE VE EL TABLERO: El tablero tiene 10 filas y 10 columnas numeradas del 1 al 10. El punto (.) indica una casilla vacía. Debajo del tablero aparece el nombre y carné del estudiante:

```
   1 2 3 4 5 6 7 8 9 10
1  . . . . . . . . . .
2  . . . . . . . . . .
3  . . . . . . . . . .
4  . . . . . . . . . .
5  . . . . . . . . . .
6  . . . . . . . . . .
7  . . . . . . . . . .
8  . . . . . . . . . .
9  . . . . . . . . . .
10 . . . . . . . . . .

[Bryan Alejandro Osoy Montesorro] - [7690 25 15671]
```

- El número de la izquierda es la FILA (del 1 al 10).
- El número de arriba es la COLUMNA (del 1 al 10).
- Para colocar tu símbolo en el centro escribes fila 5 y columna 5.
- El punto (.) significa que la casilla está libre

COMO HACER UNA JUGADA: En cada turno el programa muestra de quién es el turno y un pequeño menú:

```
Turno de: ALE [O]
1. Hacer jugada
2. Guardar partida
3. Salir al menu principal
Opcion:
```

1 **Hacer jugada**

El programa pedirá el número de fila (1 al 10) y luego el número de columna (1 al 10). Si la casilla está libre, coloca tu símbolo y pasa el turno al otro jugador.

2 **Guardar partida**

Guarda el estado actual en el archivo 'partida_guardada.txt'. Solo se guarda una partida a la vez — si ya había una guardada, se sobre escribe.

3 **Salir al menú**

Regresa al menú principal. El programa preguntará si deseas guardar la partida antes de salir.

- Si escribes una casilla ya ocupada: el programa avisa 'Casilla ocupada o fuera de rango' y te pide otra.
- Si escribes un número fuera del rango 1-10: el programa avisa y repite la pregunta.
- Si escribes una letra en vez de número: el programa avisa 'Ingresa números validos'.

¿COMO SE GANA?: Gana el jugador que logre colocar 3 símbolos consecutivos en línea recta. Hay 4 formas de ganar:

→ **Horizontal**
3 símbolos seguidos en la misma fila.

↓ **Vertical**
3 símbolos seguidos en la misma columna.

↘ **Diagonal ↘**
3 símbolos en diagonal hacia abajo-derecha.

↙ **Diagonal ↙**
3 símbolos en diagonal hacia abajo-izquierda.

Cuando alguien gana, el programa muestra el tablero final y el mensaje de victoria:

```
*** ALE ha ganado! Felicidades! ***
```

Si el tablero se llena completamente sin que nadie gane, el resultado es EMPATE y aparece el mensaje:

¡Empate! El tablero está lleno.

GUARDAR Y RESTAURAR PARTIDA

GUARDAR UNA PARTIDA: Durante el juego, en el menú de turno elige la opción 2. El programa guarda todo en un archivo llamado 'partida_guardada.txt' en la misma carpeta del programa:

```
Turno de: CARLOS [X]
1. Hacer jugada
2. Guardar partida
3. Salir al menu principal
Opcion: 2
Partida guardada correctamente!
```

- Solo se puede tener UNA partida guardada a la vez.
- Si guardas de nuevo, la partida anterior se sobre escribe.
- El archivo guarda: nombres, símbolos, puntajes, turno y el estado del tablero.
- Cuando una partida termina (victoria o empate) el archivo se elimina automáticamente.

RESTAURAR PARTIDA GUARDADA: En el menú principal elige la opción 2. El programa carga automáticamente la partida guardada y continúa desde donde se dejó:

```
--- MENU PRINCIPAL ---
1. Nuevo juego
2. Restaurar partida guardada
3. Ver puntajes
4. Salir
Elige una opcion: 2
Partida restaurada! Continua jugando...
```

El tablero aparecerá exactamente como estaba cuando se guardó, con el mismo turno y los mismos puntajes.

Si no hay ninguna partida guardada, el programa mostrará:

- 'No hay ninguna partida guardada.'

JUGAR OTRA RONDA: Al terminar una partida (**victoria o empate), el pro**grama pregunta si desean jugar otra ronda con los mismos jugadores:

```
Desean jugar otra partida con los mismos jugadores? (s/n): S
```

s (sí) → El tablero se limpia y empieza una nueva ronda. Los puntajes se conservan.

n (no) → Regresa al menú principal.

ERRORES COMUNES Y SOLUCIONES:

Problema	Solución
Escribo una letra y dice 'número no válido'	Solo se aceptan números en ese campo. Escribe un número del 1 al 10.
Dice 'Casilla ocupada o fuera de rango'	Esa casilla ya fue jugada o el número está fuera del rango 1-10. Elige otra posición.
El símbolo dice que ya está en uso	El Jugador 2 no puede usar el mismo símbolo que el Jugador 1. Elige un carácter diferente.
El punto (.) no me lo acepta como símbolo	El punto está reservado para las casillas vacías del tablero. Usa otro carácter como X, O, *, etc.
Dice 'No hay ninguna partida guardada'	Primero debes jugar y guardar una partida (opción 2 durante el juego) antes de restaurarla.

EJEMPLO DE UNA PARTIDA

Paso1: Elegir Nuevo Juego

```
=== BIENVENIDO AL TOTITO (3 EN RAYA) ===
Tablero de 10x10

--- MENU PRINCIPAL ---
1. Nuevo juego
2. Restaurar partida guardada
3. Ver puntajes
4. Salir
Elige una opcion: 1

--- NUEVO JUEGO ---
Nombre del Jugador 1: ALE
Simbolo para ALE (X, O u otro caracter): O
Nombre del Jugador 2: CARLOS
Simbolo para CARLOS (X, O u otro caracter): X
```

Paso 2: El tablero aparece y ALE hace su jugada:

```
Turno de: ALE [O]
1. Hacer jugada
2. Guardar partida
3. Salir al menu principal
Opcion: 1
Fila (1-10): 1
Columna (1-10): 1

   1 2 3 4 5 6 7 8 9 10
1  O . . . . . . . . .
2  . . . . . . . . . .
3  . . . . . . . . . .
4  . . . . . . . . . .
5  . . . . . . . . . .
6  . . . . . . . . . .
7  . . . . . . . . . .
8  . . . . . . . . . .
9  . . . . . . . . . .
10 . . . . . . . . . .

[Bryan Alejandro Osoy Montesorro] - [7690 25 15671]
```

Paso 3: Guardar y Continuar después:

```
Turno de: CARLOS [X]
1. Hacer jugada
2. Guardar partida
3. Salir al menu principal
Opcion: 3
Deseas guardar antes de salir? (s/n): S
Partida guardada correctamente!
```

```
Main.class
Main.java
partida_guardada.txt
```

Paso 4: Restaurar partida Guardada:

```
--- MENU PRINCIPAL ---
1. Nuevo juego
2. Restaurar partida guardada
3. Ver puntajes
4. Salir
Elige una opcion: 2
Partida restaurada! Continua jugando...
```

Paso 5: Ganador de la partida:

```
   1 2 3 4 5 6 7 8 9 10
1  O . . . . . . . . .
2  O X . . . . . . . .
3  O . . . . . . . . .
4  . . . . . . . . . .
5  . . . . . X . . . .
6  . . . . . . . . . .
7  . . . . . . . . . .
8  . . . . . . . . . .
9  . . . . . . . . . .
10 . . . . . . . . . .

[Bryan Alejandro Osoy Montesorro] - [7690 25 15671]

*** ALE ha ganado! Felicidades! ***
```

FLUJO COMPLETO DEL PROGRAMA

Paso	Acción del usuario	Lo que hace el programa
1	Ejecuta java Main	Muestra bienvenida y el menú principal con 4 opciones.
2	Elige opción 1 (Nuevo juego)	Solicita el nombre del Jugador 1.
3	Escribe nombre de J1	Solicita el símbolo para J1. Valida que no sea '.' y que tenga al menos 1 carácter.
4	Elige símbolo para J1 (ej: X)	Solicita el nombre del Jugador 2.
5	Escribe nombre de J2	Solicita símbolo para J2. Valida que sea diferente al de J1.
6	Elige símbolo para J2 (ej: O)	Limpia el tablero y muestra la cuadrícula 10×10 con el nombre y carnet del estudiante debajo.
7	Elige opción 1 (Hacer jugada)	Solicita el número de fila (1–10) y el número de columna (1–10).
8	Ingresa fila y columna	Coloca el símbolo del jugador actual en esa posición. Verifica si hay ganador o empate.
9	(durante el juego) Elige opción 2	Guarda toda la partida en 'partida_guardada.txt' y continúa jugando.
10	Alguien gana o hay empate	Muestra el resultado, actualiza puntajes y pregunta si desean jugar otra ronda.
11	Menú principal → opción 2	Restaura la partida guardada y continúa desde exactamente el mismo punto.

CONCLUSIONES

1. Importancia de la POO

La Programación Orientada a Objetos demostró ser fundamental para organizar el código de forma clara y mantenible. Al dividir el sistema en clases con responsabilidades específicas (Jugador, Tablero, Partida, Main), cada parte del programa es independiente y fácil de entender sin necesidad de revisar todo el código completo.

2. Persistencia con archivos

El uso de archivos de texto plano para guardar y restaurar partidas resultó ser una solución sencilla y efectiva. Las clases FileWriter, BufferedWriter, FileReader y BufferedReader de Java permiten manejar archivos de forma confiable sin dependencias externas.

3. Complejidad del tablero ampliado

Ampliar el tablero de 3×3 a 10×10 aumentó considerablemente la complejidad del algoritmo de detección de ganadores. En un tablero de 3×3 hay pocas combinaciones posibles, pero en 10×10 el método hayGanador() debe revisar cientos de combinaciones en cuatro direcciones.

4. Validación de entradas

Durante el desarrollo se aprendió la importancia crítica de validar todos los datos ingresados por el usuario. Sin validaciones adecuadas el programa puede lanzar excepciones como NumberFormatException (al ingresar letras donde se esperan números) o ArrayIndexOutOfBoundsException (al ingresar coordenadas fuera de +rango).

5. Reutilización de objetos

Al mantener las instancias de Jugador entre rondas del mismo juego, fue posible conservar el puntaje acumulado sin necesidad de crear nuevos objetos, lo que demuestra cómo la reutilización de objetos optimiza el uso de memoria.